

Implementação de uma Arquitetura Lambda para Processamento de Dados em Batch e Tempo Real

Trabalho de Graduação

Caio Bonani Carvalho
Orientadora: Prof. Dra. Daniela Leal Musa

Universidade Federal de São Paulo – UNIFESP
Instituto de Ciência e Tecnologia

Dezembro de 2025



Agenda

- 1 Introdução
- 2 Objetivos
- 3 Fundamentação Teórica
- 4 Metodologia e Tecnologias
- 5 Validação e Testes
- 6 Cronograma
- 7 Conclusão

Contexto: O Desafio do Big Data

- **Cenário Atual:** Geração exponencial de dados (IoT, Redes Sociais, Logs).
- **Os 5 Vs:** Volume, Velocidade, Variedade, Veracidade e Valor.
- **Dicotomia de Requisitos:**
 - **Tempo Real (Stream):** Respostas imediatas, baixa latência (ex: alertas de falha).
 - **Histórico (Batch):** Análises profundas, alta precisão, grandes volumes (ex: tendências anuais).
- **Limitações Tradicionais:**
 - RDBMS e arquiteturas monolíticas sofrem com escalabilidade.

Problema:

Como conciliar a necessidade de ingestão de alta velocidade com a necessidade de reprocessamento histórico confiável e imutável?

- Sistemas baseados apenas em atualizações destrutivas (CRUD) são frágeis a erros humanos.
- Necessidade de uma arquitetura que suporte **imutabilidade** e **recomputação**.

Objetivo Geral:

- Projetar, implementar e avaliar uma **Arquitetura Lambda** utilizando ferramentas *open-source*.

Objetivos Específicos:

- 1 Estabelecer pipeline de ingestão desacoplado.
- 2 Implementar Camada de Velocidade para baixa latência.
- 3 Estruturar Camada de Lote imutável.
- 4 Integrar Camada de Serviço para consultas unificadas.
- 5 Validar resiliência e performance.

Uma abordagem híbrida composta por três camadas:

- **Batch Layer:**

- Gerencia o *Master Dataset* (imutável).
- Pré-calcula *Batch Views* (V_b).
- Alta latência, alta precisão.

- **Speed Layer:**

- Processa dados recentes.
- Gera *Real-time Views* (V_s).
- Baixa latência, consistência eventual.

- **Serving Layer:**

- Unifica as visões para consulta ($Q = V_b \oplus V_s$).

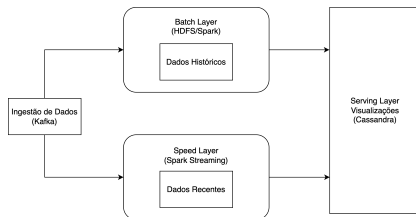


Figura: Visão Geral da Arquitetura Lambda.

A implementação utiliza um ecossistema *open-source* consolidado:

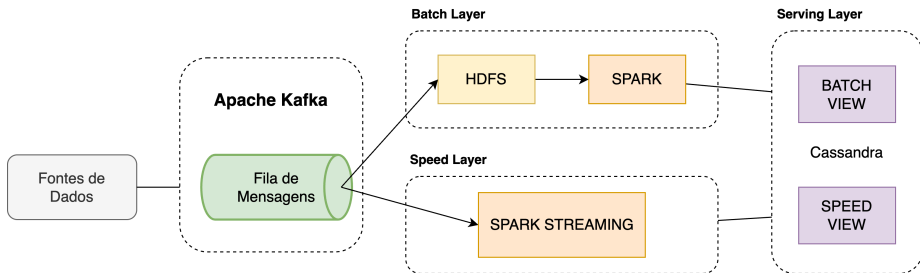


Figura: Arquitetura Lambda.

- **Ingestão:** Apache Kafka (Desacoplamento e Buffer).
- **Processamento:** Apache Spark (Unificado para Batch e Streaming).
- **Armazenamento:** HDFS (Master Dataset).
- **Serviço:** Apache Cassandra (Leituras rápidas).
- **Infraestrutura:** Docker (Orquestração).

Detalhe: Speed Layer

- **Tecnologia:** Spark Structured Streaming.
- **Funcionamento:**
 - Leitura contínua do Kafka.
 - Micro-batches de 1 segundo.
 - Escrita incremental no Cassandra.
- **Foco:** Latência mínima e alertas em tempo real.



Figura: Pipeline Speed Layer.

Detalhe: Batch Layer

- **Tecnologia:** Spark Core + HDFS.
- **Funcionamento:**
 - Armazenamento dos dados brutos no Master Dataset.
 - Leitura completa do HDFS + Reprocessamento.
 - Correção de dados errados da Speed Layer.
 - Substituição das views no Cassandra.
- **Foco:** Precisão e completude.



Figura: Pipeline Batch Layer.

Cenário de uso: Dados públicos do **SUS** (DATASUS).

Testes de Robustez:

Cenário	Objetivo
Queda de Partição Kafka	Verificar recuperação sem perda de dados.
Queda de Spark Worker	Testar redistribuição de tarefas e continuidade.
Reprocessamento Batch	Validar consistência final após falhas na Speed Layer.

Métricas:

- **MTTR** (Mean Time To Recovery) – tempo que o sistema demora para voltar ao funcionamento após falhas induzidas.
- Latência ponta-a-ponta – tempo total de processamento.
- Throughput (Vazão) – capacidade de processar um grande volume de dados.

Monitoramento

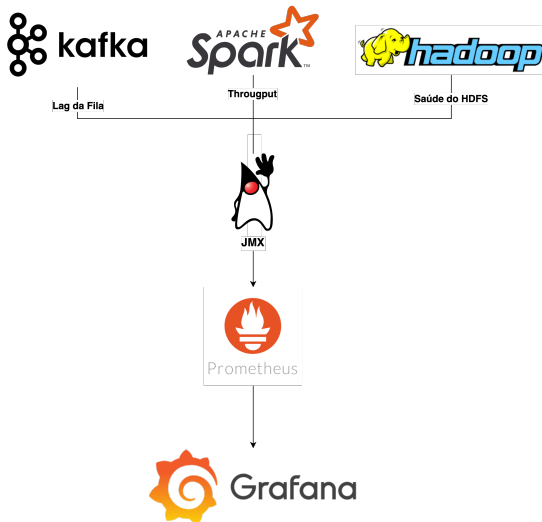


Figura: Arquitetura de Observabilidade (Prometheus + Grafana).

- **TCC I (Concluído):**

- Definição do problema.
- Revisão Bibliográfica.
- Análise da arquitetura.
- Análise de ferramentas.
- Escrita da Monografia.

- **TCC II (Próximos Passos):**

- Construção do ambiente (Infraestrutura).
- Configuração dos componentes da arquitetura.
- Criação das massas de teste.
- Execução dos experimentos e coleta de métricas.
- Escrita final.

- A Arquitetura Lambda oferece um modelo robusto para lidar com a complexidade do Big Data moderno.
- A separação de responsabilidades (Batch vs Speed) permite otimizar simultaneamente para latência e precisão.
- O uso de ferramentas *open-source* (Kafka, Spark, Cassandra) democratiza o acesso a arquiteturas de nível empresarial, sem a necessidade de investimento em soluções proprietárias.
- **Contribuição Esperada:** Um guia de referência prático e validado para implementação de pipelines híbridos resilientes.

Obrigado!

caio.bonani@unifesp.br